

SDP



Copilot Plugins & APM

Composable AI Extensions

Install once, configure everywhere

"How do I extend Copilot and share that configuration across every team clone?"

Every team has custom instructions, skills, and integrations — but configuration scatters across developer machines.

APM turns your agent setup into infrastructure that travels with the repo.

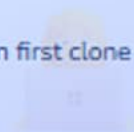
Platform Engineers

Standardize Copilot config across every service repo



Developers

Inherit the team's full AI context on first clone



Enterprise Architects

Govern the baseline while teams extend it safely



Manual setup overhead

New contributors spend hours configuring plugins before writing a line of code

Configuration drift

Developer A and Developer B have different plugin versions by Friday

Plugin proliferation

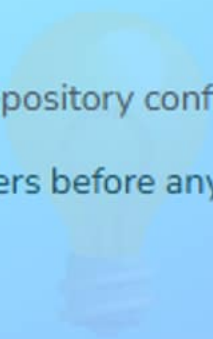
700+ extensions in the VS Code marketplace — no vetted fast-path

PART 1

The Opportunity

Per-developer config → per-repository config

Why the packaging shift matters before any CLI syntax

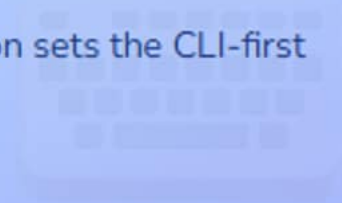


PART 2

CLI-First Plugin Management

Browse, install, and explore from the terminal

Live ecosystem exploration sets the CLI-first tone



PART 3

Building an APM Manifest

apm.yml + lockfile as team infrastructure

Codify configuration; add governance companion beat

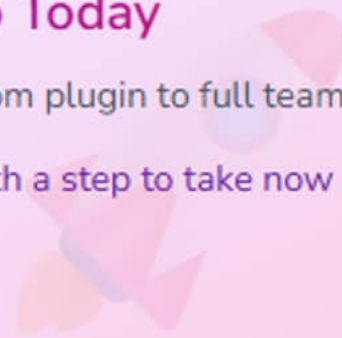


PART 4

What You Can Do Today

Concrete action ladder from plugin to full team

Every attendee leaves with a step to take now



Click any section to jump directly there

The Opportunity

Frame the paradigm shift — per-developer → per-repository configuration — before any CLI commands appear.



Paradigm Shift

From scattered machines to versioned
repo config



Ecosystem Ready

Marketplace + community catalog
waiting to explore



Zero Onboarding

git clone && apm install = full AI
context

The question teams are asking:

↳ `git clone` → full Copilot context in one command?

copilot plugin vs APM — Same Ecosystem, Different Jobs

copilot plugin

Personal exploration

Install, try, remove — no team impact

Immediate activation — no manifest needed

30-second workflow

copilot plugin install <name>

Scoped to your user account

APM (apm install)

Team infrastructure

apm.yml committed alongside code

Lockfile pins exact versions for everyone

One-command onboarding

git clone && apm install

Plugins, instructions, skills, MCP — one pass

Per-Developer Configuration → Per-Repository Configuration

Per-Developer (Before)

Setup instructions live in a wiki page

Manual configuration

Every new clone → every developer does it again

Plugin versions drift silently across machines

AI context is personal, not shared

Per-Repository (After)

apm.yml committed next to the code

Automatic hydration

git clone && apm install — done

Lockfile enforces identical versions for all

AI context is team knowledge, version-controlled

Four Capabilities Unlocked Today



Marketplace Discovery

apm marketplace browse — vetted plugins from the CLI without leaving your terminal



One Manifest, Everything

apm.yml declares plugins, instructions, skills, and MCP servers in a single versioned file



Lockfile Reproducibility

apm-lock.yml pins exact versions — identical AI config on every machine, every time



Cross-Editor Portability

Plugins work in VS Code, Copilot CLI, and any ACP-compatible client — install once, use everywhere

CLI-First Plugin Management

Live-demo apm marketplace browse and copilot plugin install; hands-on ecosystem contact that sets the CLI-first tone.



Marketplace Browse

Find vetted plugins without leaving the terminal



One-Command Install

copilot plugin install — immediate activation



Update & Remove

Full lifecycle management from the same CLI

From exploration to team config:
↳ copilot plugin install → lock into apm.yml

apm marketplace browse — Vetted Extensions at the Terminal

terminal

```
# List all vetted plugins
apm marketplace browse

# Filter by category
apm marketplace browse --category code-review
apm marketplace browse --category testing

# Search by keyword
apm marketplace browse --search security

# See what you have installed
copilot plugin list
```



Category Filtering

Narrow by code-review, testing, integrations, and more



Keyword Search

Find plugins by name, description, or capability



Inventory Check

copilot plugin list shows installed versions and scope

From Browse to Running — Under 60 Seconds

No UI navigation, no extension marketplace search, no restart required

```
$ apm marketplace browse --category code-review
```

```
code-review-assistant v1.2.3 Review PRs for security, style, and architecture  
test-coverage-guard v0.8.1 Flag PRs that reduce test coverage below threshold  
dependency-auditor v1.0.0 Surface vulnerable or outdated dependencies in PRs
```

```
$ copilot plugin install code-review-assistant
```

⌚ Installing:

- ✓ Resolved code-review-assistant@1.2.3
- ✓ Plugin registered with Copilot runtime
- ✓ Available in VS Code, CLI, and ACP clients immediately

✓ Plugin active — no restart required

✓ Next: lock it into apm.yml for the whole team

30 seconds from browse to running

Where to Find Plugins Before You Build



Official Marketplace

apm marketplace browse —
GitHub and Microsoft
maintained, vetted, and signed

First-party quality bar

Category and keyword search

Contribution guide for new plugins



Awesome GitHub Copilot

Community-curated list of
plugins, skills, MCP servers, and
workflows

Third-party and niche integrations

Skills files and instruction
templates

Community-validated use cases



github/copilot-plugins

First-party source code, manifest
schema docs, and plugin
contribution guidelines

Build your own plugin here

Official schema reference

Issue tracker for bugs and requests

copilot plugin for Exploration, APM for Infrastructure

Use copilot plugin when...

You are evaluating a plugin personally

Ad-hoc capability test

Try it, discard it, no team impact

You want immediate activation without a manifest

Personal customization outside any repo

Graduate to APM when...

The plugin proves valuable for the team

Commit apm.yml

Lock it in so every clone gets it

You want CI to validate the lockfile

Different repos need different plugin profiles

Building an APM Manifest

Walk a complete `apm.yml`, close with the lockfile capstone, then add the bounded managed-settings governance companion.



apm.yml Anatomy

Plugins, instructions, skills, MCP in one file



Lockfile Pattern

`apm-lock.yml` treats AI config like package-lock



Managed Settings

Enterprise floor — separate from `apm.yml`, additive

The lockfile is non-negotiable:
↳ `works-on-my-machine` impossible for AI config

apm.yml — Everything Your Team's Agent Needs

apm.yml

```
version: 1
```

```
plugins:
```

- name: code-review-assistant
version: ^1.0.0
- name: test-generator
version: ^2.1.0

```
instructions:
```

- path: .github/copilot-instructions.md

```
skills:
```

- path: .github/skills/testing/SKILL.md

```
mcpServers:
```

- name: github-mcp
source: npm:@modelcontextprotocol/server-github
version: ^1.2.0

Plugins

Version-ranged dependencies resolved at apm install time

Instructions + Skills

Instructions and skills files hydrated alongside plugins

MCP Servers

Live integrations from npm, GitHub, or local paths

Reproducible

One apm install gives every clone identical AI context

apm-lock.yml — Treat AI Config Like package-lock.json

Without Lockfile

^1.0.0 resolves differently on Monday vs Friday
New plugin release = different behavior

Developer A gets 1.2.3, Developer B gets 1.3.0

No review record
Plugin upgrades happen silently

Debugging: which version are you running?

With apm-lock.yml

1.2.3 pinned for everyone
Exact version, hash, and download URL frozen

apm install always produces identical results

Lockfile diff in every PR
Plugin upgrades are explicit, reviewable changes

apm install --frozen-lockfile fails if drifted

100%

version consistency

0

silent upgrades

1 PR

per plugin change



Commit both apm.yml and apm-lock.yml — they are a pair, not a choice.

APM Packages Config — Managed Settings Govern the Floor

APM (apm.yml)

Project-scoped packaging

Lives in the repo, travels with the code

Plugins, instructions, skills, MCP — one manifest

Team installs via apm install

Every clone gets identical AI context

Teams own their config — no enterprise dependency

```
# apm.yml — per-repository
plugins:
  - name: code-review-assistant
```

Managed Settings

Enterprise-wide governance

Additive floor beneath team customization

enabledPlugins + extraKnownMarketplaces — additive keys

Teams extend via copilot/teams/

Least-restrictive merges; enterprise is non-overridable

VS Code, CLI, Copilot App, cloud agent — Business/Enterprise

```
// team-mappings.json
{ "backend-team": "teams/backend.json" }
```

git clone && apm install

The complete Copilot onboarding — no wiki page, no Slack message, no manual steps

0

manual setup steps for a new contributor to get
full AI context

🔧 Plugins installed

Exact versions from lockfile, identical to every teammate

📄 Instructions loaded

Team conventions and custom context ready immediately

🧠 Skills available

Domain-specific workflows active from the first session

🌐 MCP servers running

Live integrations wired — GitHub, AWS docs, whatever the team uses

💡 APM packages reproducible project configuration. This is infrastructure — commit it.

What You Can Do Today

Concrete action ladder: one plugin, one apm.yml commit, one lockfile diff — all doable within the hour.



Try Locally

One plugin in 15 minutes, no team approval needed



First Manifest

One repo, one apm.yml, commit and share



Lockfile PRs

Review diff on the next plugin update

The complete onboarding:

```
↳ git clone && apm install – zero manual steps
```

From First Plugin to Team Infrastructure

Each stage unlocks the next — start anywhere, the path is linear



L2 is the leverage point — one committed apm.yml changes the onboarding story for everyone

Real Patterns Teams Are Running Now



Standardized Reviews

code-review-assistant in every repo — new hires get consistent review on first clone, no setup doc

One apm.yml template

Lockfile in every repo

0 manual onboarding steps



Multi-Project Dev

Three repos, three apm.yml files — each project gets purpose-fit plugins on apm install

Context switches with cd

No global plugin sprawl

Each repo owns its config



CI Lockfile Guard

PR CI runs --frozen-lockfile — plugin upgrades become explicit, reviewable like any dependency bump

apm install --frozen-lockfile

Fails on any drift

Plugin changes need PR review

From Scattered Config to Reproducible Infrastructure

Before

Manual plugin setup docs in a wiki

Different plugin versions per developer

New hire spends hours on Copilot setup

No review process for plugin changes

After

apm.yml commits live alongside code

apm-lock.yml pins exact versions team-wide

git clone && apm install — zero manual steps

Lockfile diffs reviewed in every PR

0

manual setup steps for new contributors

100%

version consistency across all team
machines

1 PR

to review and approve any plugin
change

✓ What You Can Do Today

Today

- ❑ Run apm marketplace browse in your terminal
- ❑ Install one plugin: copilot plugin
install <name>
- ❑ Test it in Copilot Chat or CLI immediately

This Week

- ❑ Create apm.yml in one repo you own
- ❑ Run apm install and commit the lockfile
- ❑ Have a teammate clone and verify identical setup

This Month

- ❑ Add CI lockfile validation step to one pipeline
- ❑ Expand apm.yml with instructions and skills
- ❑ Audit team plugins — standardize across repos

🔑 Key Takeaway

Configuration as code starts with a single apm.yml commit — everything else follows from that habit.

 Official Documentation[Agent plugins for Copilot customization](#)

Plugin concepts, installation, and VS Code integration

[APM - Agent Package Manager](#)

Manifest schema, lockfile mechanics, and CLI reference

[Official Copilot Plugins Repository](#)

First-party plugins, manifest schema, and contribution guide

 Related Talks[Copilot Primitives](#)

Instructions, skills, and MCP servers that APM orchestrates

[Copilot CLI](#)

Plugin management from the command line in depth

 Community & Ecosystem[Awesome GitHub Copilot](#)

Community-curated plugins, skills, MCP servers, and workflows

[Enterprise team specialization for managed settings](#)

Additive team plugin settings, enterprise precedence, supported clients



Copilot Plugins & APM

Composable AI Extensions — Install once, configure everywhere

apm.yml

Infrastructure: commit it with your code,
not in a wiki page

apm-lock.yml

Non-negotiable: no more works-on-my-
machine AI config

git clone && apm install

Complete onboarding: zero manual steps,
full AI context

What plugin or capability would you lock into your team's apm.yml first?